

SATQUERY AI — MASTER IMPLEMENTATION ROADMAP

0. NORTH STAR

Final objective

Build an **ISRO-oriented, EO-native, agentic analytical system**:

Natural-language query



Understand intent



Inspect available EO data



Determine required evidence



Construct workflow



Select best specialist models/tools



Execute in parallel where possible



Fuse results



Verify results



Quantify / georeference



Return answer + map + evidence + confidence



Expose execution summary

Your uploaded blueprint already points toward this shift from isolated pipelines to a dynamic multi-task system.

1. WHAT SATQUERY IS — AND IS NOT

SatQuery is

A. Natural-language EO interface

User speaks normally.

B. Query-to-workflow engine

The system determines what analytical process is necessary.

C. Specialist orchestrator

It chooses appropriate VLMs, temporal models, SAR models, segmentation models and GIS tools.

D. EO reasoning layer

It reasons across:

- optical
- multispectral
- SAR
- temporal pairs
- possibly other modalities later.

E. Geospatial execution engine

It performs actual:

- raster operations
- spectral indices
- spatial relationships
- measurements
- coordinate transformations
- vectorization.

F. Verification engine

It checks whether model outputs are spatially, temporally, numerically and sensor-wise consistent.

G. Actionable-output system

The result isn't just prose.

It can be:

- text
- map
- bbox
- polygon
- mask
- coordinates
- statistics
- GeoJSON
- execution summary.

Your technical documents already specify the transition to georeferenced GeoJSON using the original spatial transform/CRS information.

2. THE MOST IMPORTANT ARCHITECTURAL CHANGE

Your current technical specification proposes a router approximately like:

"flood" / "radar" → SAR

"change" / "before" → temporal

"count" / "ship" → optical

That is useful as a prototype, but **do not make this the final architecture.**

The uploaded specification itself currently describes this keyword-routing mechanism.

Why?

Because a query such as:

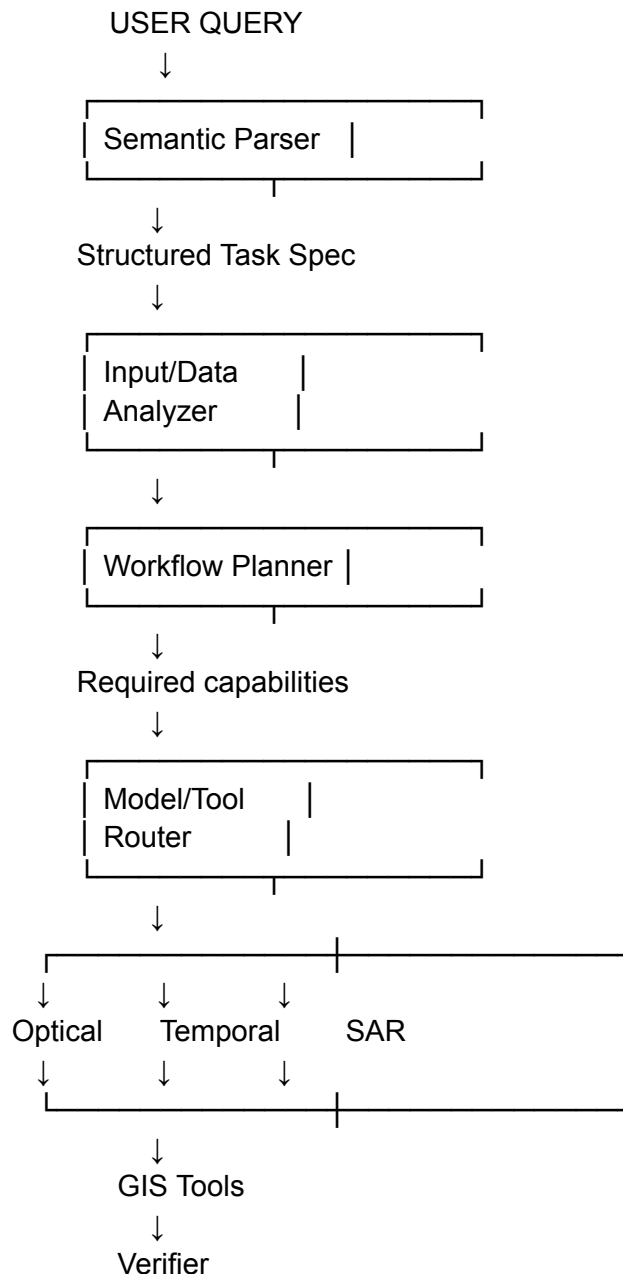
“Find newly constructed buildings in flooded areas between 2024 and 2026 using available imagery.”

contains **multiple simultaneous intents.**

A keyword router may select only one stream.

Replace it with:

Semantic Query Planner + Capability Registry + Workflow Planner

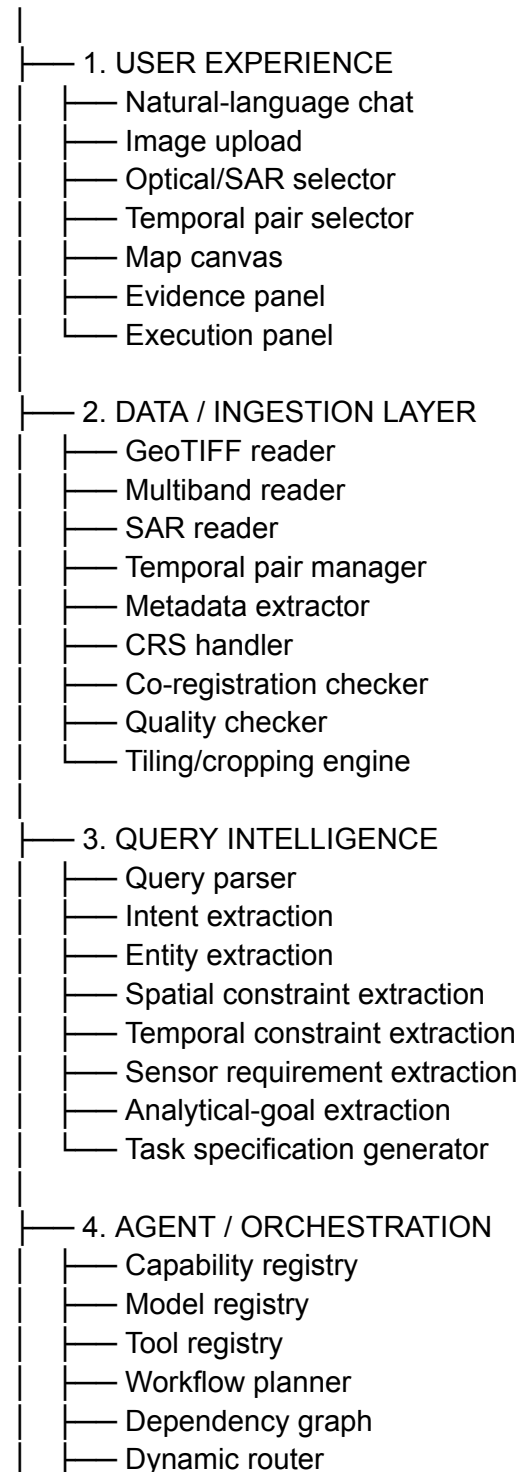


This is much closer to the agentic workflow direction we've identified in the literature.

RemoteReasoner is especially important here: its published objective is explicitly to construct geospatial reasoning workflows and autonomously explore inference paths rather than being restricted to predefined task sequences.

3. MASTER SYSTEM HIERARCHY

SATQUERY AI



- Parallel executor
- Retry/recovery
- Cost/latency manager

5. RS AI SPECIALISTS

- VQA
- Captioning
- Open-vocabulary detection
- Grounding
- Segmentation
- Change detection
- Temporal reasoning
- Optical-SAR
- EO foundation models

6. EO ANALYTICS ENGINE

- NDVI
- NDWI
- NDBI
- spectral statistics
- area
- distance
- buffer
- intersection
- proximity
- zonal statistics

7. EVIDENCE ENGINE

- spatial evidence
- temporal evidence
- sensor evidence
- model evidence
- provenance

8. VERIFICATION

- geometry consistency
- topology checks
- temporal consistency
- cross-sensor consistency
- numerical consistency
- duplicate removal
- contradiction detection

9. UNCERTAINTY

—	calibrated confidence
—	model disagreement
—	insufficient evidence
—	OOD detection
—	abstention
10. OUTPUT	
—	Natural-language answer
—	Map
—	BBoxes
—	Masks
—	GeoJSON
—	Statistics
—	Evidence
—	Confidence
—	Execution summary

4. PHASE 0 — REQUIREMENT FREEZE

Duration: 1–2 days

Before implementation, create the **SatQuery Requirements Baseline**.

Separate:

Mandatory

Explicit SIH26167 requirements.

Strongly recommended

Necessary for a strong solution.

Innovation

Our additional differentiators.

Future

Not needed for SIH MVP.

This is important because your current documents mix required features with proposed architecture.

5. PHASE 1 — ISRO TASK MODEL

Duration: 2–3 days

Construct the **ISRO Task Taxonomy**.

Track A — Single image

VQA
caption
count
detect
ground
segment

Track B — Bi-temporal

T1 + T2
↓
change
↓
change type
↓
change location
↓
change magnitude

Track C — Optical + SAR

Optical
+
SAR
↓
cross-modal reasoning

Track D — Composite

single
+
temporal
+
spatial
+
sensor
+
GIS

This final category is where we test whether the agent actually works.

6. PHASE 2 — BUILD THE MODEL REGISTRY

Duration: 2–4 days

Your prior model research already gives us the candidate universe: Earth-OneVision, RemoteReasoner, TerraMind, SkySense family, Prithvi-EO, MIGRANT, GeoGround, Think2Seg-RS, VHM, EarthGPT-X, SkyEyeGPT, GeoChat, etc.

Create:

ModelRegistry

- |
- | — VQA models
- | — Grounding models
- | — Segmentation models
- | — Temporal models
- | — SAR models
- | — Multimodal models
- | — Foundation models

Every model profile contains:

name
version
capabilities
modalities
input requirements

output types
resolution
parameter count
VRAM
latency
license
checkpoint
benchmark score
known weaknesses
preferred tasks
fallback models

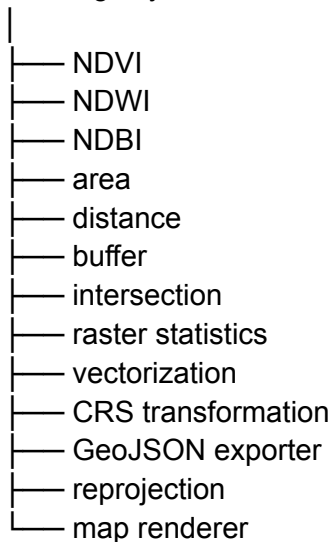
7. PHASE 3 — BUILD THE TOOL REGISTRY

Duration: 2–3 days

Model registry alone isn't enough.

Create:

ToolRegistry



The key principle:

LLM decides what operation is required. Deterministic code performs the operation.

Your uploaded architecture already proposes this separation using Rasterio/GeoPandas for objective numerical operations.

8. PHASE 4 — DATA INGESTION

Duration: 4–7 days

Build the ingestion layer first because everything depends on trustworthy input.

Supported inputs

At minimum:

GeoTIFF
multiband GeoTIFF
SAR raster
paired images
temporal images

Your technical specification proposes `.tif`, `.nc`, `.hdr`, native geotransforms, metadata, CRS and full spectral layers.

Pipeline

INPUT

↓

Format detection

↓

Metadata extraction

↓

CRS extraction

↓

Band discovery

↓

Quality check

↓

Cloud/no-data check

↓

Spatial extent

↓

Temporal metadata



Co-registration check



Canonical internal representation

9. PHASE 5 — CANONICAL EO DATA MODEL

Duration: 2–3 days

Do not allow individual models to consume arbitrary raw files.

Convert every input into a common internal structure.

Example:

```
{
  "scene_id": "scene_001",
  "sensor": "optical",
  "platform": "Cartosat-2S",
  "acquisition_time": "...",
  "crs": "...",
  "resolution_m": 1.0,
  "bands": [...],
  "bbox": [...],
  "geotransform": [...],
  "quality": {...}
}
```

For temporal:

```
{
  "pair_type": "bi_temporal",
  "t1": {...},
  "t2": {...},
  "registration_error": 0.42
}
```

For optical/SAR:

```
{
```

```
"pair_type": "cross_sensor",
"optical": {...},
"sar": {...},
"coregistration": {
  "valid": true,
  "error_px": 0.7
}
}
```

10. PHASE 6 — INPUT VALIDATION

Duration: 3–5 days

This is a critical reliability layer.

Validate

Format

✓ supported

Spatial

✓ georeferenced

✓ CRS exists

Temporal

✓ T1/T2 available

✓ dates meaningful

Multimodal

✓ same geographic region

✓ co-registration acceptable

Data quality

✓ enough valid pixels

✓ acceptable cloud/no-data

✓ required bands exist.

Return a machine-readable status:

READY
PARTIALLY_READY
NEEDS_USER_INPUT
UNSUPPORTED
INSUFFICIENT_DATA

11. PHASE 7 — QUERY UNDERSTANDING

Duration: 4–7 days

Build a **structured Task Specification**.

Example query:

“Find newly constructed buildings within 500 m of the flooded region between 2024 and 2026.”

Produce:

```
{
  "primary_task": "temporal_object_spatial_analysis",
  "objects": ["building", "flood"],
  "temporal": {
    "start": "2024",
    "end": "2026"
  },
  "spatial_constraint": {
    "relation": "within",
    "distance": 500,
    "unit": "meter",
    "reference": "flooded_region"
  },
  "operations": [
    "flood_detection",
    "building_detection",
    "change_detection",
    "spatial_filter"
  ]
}
```

This becomes the contract between language and execution.

12. PHASE 8 — WORKFLOW PLANNER

Duration: 5–7 days

Transform the Task Specification into a directed graph.

QUERY



Validate inputs



Detect flood



Segment flood



Detect buildings T1



Detect buildings T2



Match buildings



Find newly appearing buildings



Create 500m buffer



Spatial intersection



Measure



Verify



Generate outputs

The planner should understand **dependencies**.

For example:

buffer(flood)

cannot execute before:

flood segmentation

13. PHASE 9 — MODEL ROUTER

Duration: 4–7 days

Now use the registry.

The router evaluates candidates on:

task suitability
+
sensor compatibility
+
resolution
+
benchmark quality
+
latency
+
VRAM
+
availability

Example:

TASK = grounding
OBJECT = aircraft
IMAGE = high-resolution optical

Candidates:

A → GeoGround
B → Grounding DINO
C → Earth-OneVision

Router chooses:

A

But:

TASK = multisensor reasoning
INPUT = optical + SAR

Router:

Earth-OneVision / TerraMind / SkySense-family candidate

Earth-OneVision is especially worth benchmarking because its 2026 paper describes a 2B RS-MLLM spanning optical, SAR, infrared, multispectral, temporal and video modalities with 9 task categories.

14. PHASE 10 — SINGLE-IMAGE ENGINE

Duration: 4–7 days

Implement first complete vertical slice.

upload image

↓

query

↓

VQA

↓

grounding

↓

SAM2

↓

GeoJSON

↓

map

This gives the team a functioning foundation.

Example

“Find all ships.”

Output:

count = 14

boxes = 14

masks = 14

coordinates = 14

GeoJSON = generated

15. PHASE 11 — ACTIVE PERCEPTION

Duration: 3–5 days

This should be an **advanced capability**, not MVP-critical.

From our previous research, this is worth pursuing because high-resolution RS imagery often contains tiny objects, and active crop/zoom inspection has emerged as an RS-VQA direction.

Workflow:

Full image
↓
Initial analysis
↓
uncertain / tiny target
↓
crop
↓
zoom
↓
specialist model
↓
re-evaluate

This can dramatically improve small-object cases.

16. PHASE 12 — GROUNDING + SEGMENTATION

Duration: 4–7 days

Your current architecture's cascade:

VLM coordinates
↓
SAM2 prompt
↓
mask

is a good direction. The uploaded technical design specifically proposes this cascade.

Implement:

text
↓
grounding
↓
bbox / point
↓
SAM2
↓
mask
↓
contour
↓
polygon
↓
georeference

17. PHASE 13 — GEOSPATIAL VECTOR ENGINE

Duration: 3–5 days

This is where your diagram becomes an actual GIS system.

Mask
↓
Contour extraction
↓
Polygon simplification
↓
Pixel → map coordinates
↓
CRS transformation
↓
GeoJSON FeatureCollection

The uploaded implementation blueprint already specifies contour extraction followed by re-projection using spatial-transform metadata.

Important

Internally maintain native CRS.

Only convert to EPSG:4326 at the presentation/export boundary where appropriate.

18. PHASE 14 — TEMPORAL ENGINE

Duration: 5–10 days

This is one of the highest priorities because **bi-temporal analysis is central to the SIH problem**.

Build:

T1
+
T2
↓
co-registration verification
↓
feature/change model
↓
change mask
↓
object matching
↓
change classification
↓
natural-language explanation

Support:

Appearance

Object exists at T2, not T1.

Disappearance

Object exists at T1, not T2.

Growth

Object/region expanded.

Reduction

Object/region decreased.

Transformation

Land-cover class changed.

19. PHASE 15 — TEMPORAL REASONING LEVELS

Don't stop at a change mask.

Implement progressively:

LEVEL 1

Where changed?

LEVEL 2

What changed?

LEVEL 3

Which objects changed?

LEVEL 4

How much changed?

LEVEL 5

What does the change represent?

LEVEL 6

What should the user inspect next?

This hierarchy comes directly from our earlier research synthesis.

20. PHASE 16 — OPTICAL + SAR ENGINE

Duration: 5–10 days

This is one of the **top two implementation priorities**.

Important distinction:

Don't do:

RGB tensor + SAR tensor
→ concatenate
→ LLM

Instead:

Optical preprocessing
+
SAR preprocessing
↓
quality assessment
↓
co-registration validation
↓
sensor-specific encoders
↓
multimodal fusion
↓
task reasoning
↓
evidence fusion

21. YOUR CUSTOM CROSS-ATTENTION BLOCK

Your technical document currently proposes:

feature_dim = 512
num_heads = 8
LayerNorm
MHA
FFN = $4 \times$ feature dimension

and cross-attention:

optical = Query
SAR = Key/Value

My recommendation:

Do **not** immediately declare this to be our final SatQuery foundation.

Use it as:

Experiment A

Then compare:

A

Early fusion

B

Late fusion

C

Cross-attention

D

Existing multimodal foundation model

E

Hybrid specialist + fusion.

Then benchmark.

Only keep the custom block if it gives a measurable benefit.

That is much more scientifically defensible.

22. PHASE 17 — SENSOR-AWARE REASONING

Duration: 3–5 days

The router should understand:

Which sensor?

Why?

How reliable?

What evidence does it contribute?

Example:

QUERY

"Map flooded area during storm."

Optical

→ cloud-contaminated

SAR

→ usable

Planner

→ prioritize SAR

→ optionally use optical as supporting evidence

Your previous research specifically identified this as a stronger target than merely supporting both modalities.

23. PHASE 18 — SPECTRAL ANALYTICS ENGINE

Duration: 3–5 days

Support:

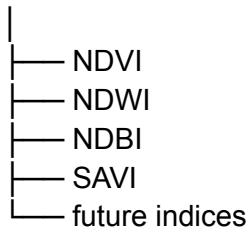
NDVI

NDWI

NDBI

But architect it generically:

IndexRegistry



Each entry:

required_bands
formula
valid_range
quality constraints
interpretation

Then:

“Find poor vegetation health.”

becomes:

language
↓
intent
↓
vegetation health
↓
select NDVI
↓
compute
↓
threshold / statistics
↓
map

Our earlier research explicitly identified this “natural language → analytical operation” pathway as more useful than simply exposing formulas.

24. PHASE 19 — EVIDENCE ENGINE

Duration: 4–7 days

This should be separate from the answer generator.

Build:

EvidenceObject

- |— source
- |— sensor
- |— timestamp
- |— geometry
- |— model
- |— confidence
- |— supporting_result
- |— quality

For an answer:

“New construction detected.”

evidence can be:

T1 absence

+

T2 presence

+

building mask

+

optical evidence

+

SAR evidence

Your uploaded architecture explicitly makes evidence fusion a central block and proposes cross-validation against analytical indices.

25. PHASE 20 — VERIFIER ENGINE

Duration: 5–7 days

This is one of the most promising ideas in your documents.

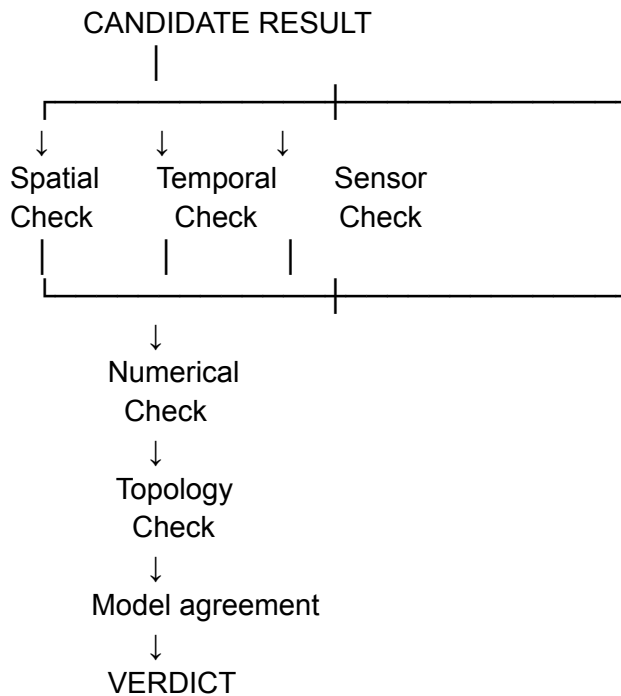
But let's improve the concept.

Your current document proposes:

model prediction → compare against physical/GIS evidence → accept/suppress.

We should turn this into:

Multi-layer Verification



26. IMPORTANT: DON'T CALL IT "PHYSICS VERIFICATION" EVERYWHERE

This is a refinement.

NDVI/NDWI can provide strong analytical evidence for some tasks, but they cannot universally certify arbitrary VLM predictions.

For example:

“This is a bridge.”

There isn't an NDWI-style physical invariant that can certify it.

So use:

Evidence & Consistency Verification Engine

with task-specific validators.

27. PHASE 21 — UNCERTAINTY ENGINE

Duration: 3–5 days

Create explicit states:

HIGH CONFIDENCE
MODERATE CONFIDENCE
LOW CONFIDENCE
INSUFFICIENT EVIDENCE
CONFLICTING EVIDENCE
UNSUPPORTED

Don't simply expose raw model probabilities.

Build calibration using validation data.

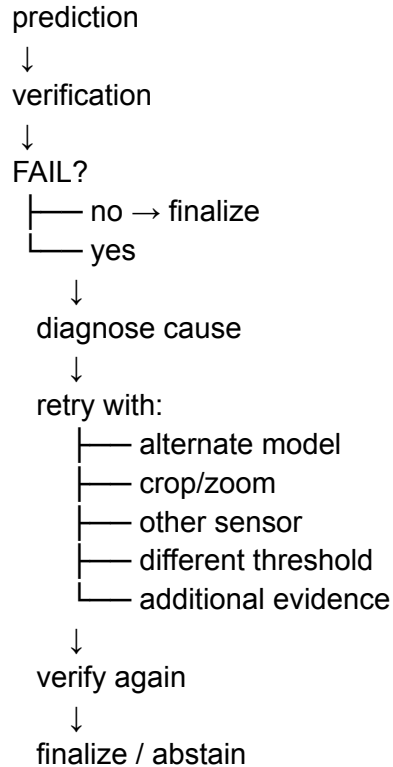
28. PHASE 22 — SELF-CORRECTION LOOP

Duration: 3–5 days

Your current concept is:

prediction
↓
verification
↓
suppress hallucination

Let's make it more useful:



Now it becomes a true **self-correction mechanism**, not merely a filter.

29. PHASE 23 — MULTI-MODEL CONSENSUS

Duration: 3–5 days

Suppose:

Grounding model → 18 objects

VLM → 17

SAM → 18 masks

Use consistency.

But don't use naive voting.

Weight based on:

task
model reliability
sensor
resolution
confidence calibration
agreement

Then generate:

supported
partially supported
conflicting

30. PHASE 24 — GEOINT OUTPUT ENGINE

Duration: 4–7 days

Every task gets a standardized response schema:

RESULT
SUMMARY
STATISTICS
MAP
GEOMETRY
EVIDENCE
CONFIDENCE
LIMITATIONS
EXECUTION

Your current API blueprint already points toward a unified JSON payload with query ID, task type, summary, processing metrics, confidence and geospatial data.

Expand it to:

```
{  
  "query_id": "...",  
  "task": "...",  
  "answer": "...",  
  "statistics": {},  
  "geospatial_outputs": [],
```

```
"evidence": [],  
"confidence": {},  
"models_used": [],  
"tools_used": [],  
"verification": {},  
"limitations": [],  
"execution": []  
}
```

31. PHASE 25 — EXECUTION TRACE

Duration: 2–4 days

Show:

Query understood

✓

Input validated

✓

Temporal analysis selected

✓

Grounding model selected

✓

GIS operation executed

✓

Verification completed

✓

Do **not** expose hidden chain-of-thought.

Expose:

What the system did.

Not:

Private reasoning text.

32. PHASE 26 — INTERACTIVE MAP

Duration: 5–7 days

The map should support:

base imagery
T1
T2
SAR
change layer
detection layer
mask layer
buffer
measurement

Interaction:

click polygon
↓
object details
↓
confidence
↓
source imagery
↓
temporal evidence

Export:

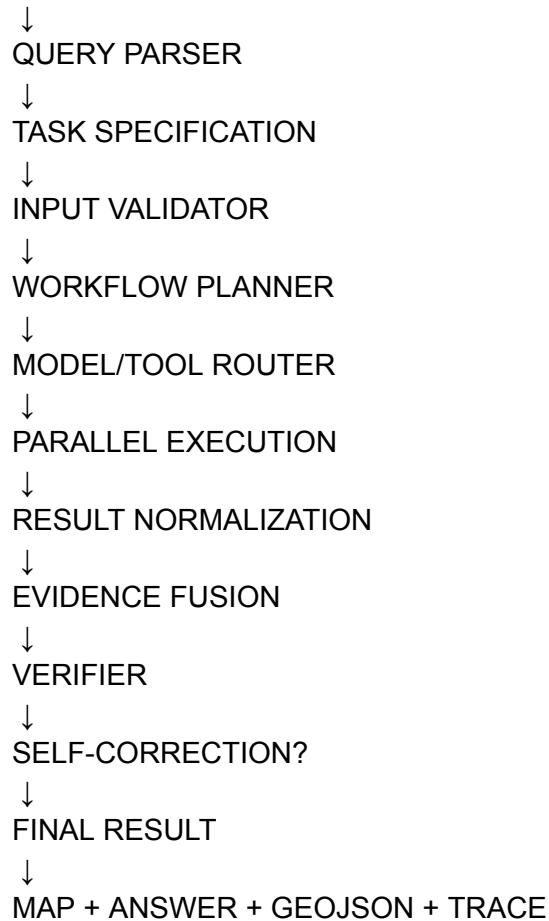
GeoJSON
KML
CSV

Your uploaded documents already propose React/Leaflet or Next.js/Mapbox-style frontends connected to GeoJSON outputs.

33. PHASE 27 — BUILD THE AGENT END-TO-END

Now we connect everything.

USER



This is the first point where we can legitimately call the whole thing:

SatQuery AI

34. PHASE 28 — BENCHMARK AGAINST EXISTING MODELS

This is critical.

Don't benchmark only:

SatQuery accuracy.

Benchmark:

Model alone
vs
Model + GIS
vs
Model + specialist
vs
SatQuery

For example:

Query

“Find buildings that appeared after 2024.”

Compare:

Earth-OneVision
RemoteReasoner
specialized change model
SatQuery

35. PHASE 29 — BUILD THE SATQUERY BENCHMARK

Create ~100–200 queries initially.

20 single-image

20 grounding

20 temporal

20 optical-SAR

20 analytical/GIS

20 complex multi-step

Then increase.

36. DIFFICULTY LEVELS

Level 1

Simple VQA.

Level 2

Object detection.

Level 3

Grounding.

Level 4

Temporal.

Level 5

Cross-sensor.

Level 6

Multi-step.

Level 7

Missing/poor data.

Level 8

Conflicting evidence.

Level 9

Complex analytical query.

Level 10

Fully compositional geospatial investigation.

37. METRICS

Visual

VQA accuracy
mAP
IoU
Dice

Grounding

P@0.5
IoU
localization error

Temporal

F1
IoU
change classification accuracy
object correspondence accuracy

Geospatial

area error %
distance error
coordinate error
polygon IoU

Agent

task-selection accuracy
tool-selection accuracy
workflow success rate
unnecessary-tool rate

Reliability

hallucination rate
false-positive rate
abstention precision
calibration error
verification success

System

latency
VRAM
throughput
failure recovery

38. PHASE 30 — ABLATION STUDIES

This is extremely important if we want the project to look technically serious.

Test:

Full system

vs

Without router

vs

Without verifier

vs

Without temporal specialist

vs

Without SAR

vs

Without GIS engine

vs

Without evidence fusion

vs

Without active perception.

Then demonstrate:

Which component actually contributes what?

39. PHASE 31 — YOUR CUSTOM CROSS-ATTENTION ABLATION

Specifically test:

No fusion

vs

concatenation

vs

late fusion

vs

cross-attention

vs

existing multimodal foundation model

Parameters to vary:

256 / 512 / 768

heads:

4 / 8 / 12

fusion depth:

1 / 2 / 4 blocks

But only if we actually have training data and compute sufficient to justify experimentation.

40. PHASE 32 — DATASET STRATEGY

Your current technical blueprint proposes RSVLM-QA, VRSBench, DIOR-RSVG, DOTA, FloodNet and LandCover.ai.

We should expand that into:

Language

- VRSBench
- RSVQA/RSVLM-QA
- Earth-OneVision-type datasets
- SkyEye instruction data where license permits.

Grounding

- DIOR-RSVG
- DOTA
- refGeo
- RSVG datasets.

Temporal

- LEVIR-CD
- WHU-CD
- CDD
- CDVQA
- temporal VLM datasets.

Multimodal

- SEN12MS
- optical/SAR datasets
- MMRS-type resources.

Segmentation

- iSAID
- SpaceNet
- FloodNet

- LoveDA.

EO foundation adaptation

- BigEarthNet
 - HLS
 - Sentinel datasets.
-

41. VERY IMPORTANT — DATA SPLIT STRATEGY

Do not randomly split geographically similar images.

We should have:

Train

Known regions.

Validation

Different scenes.

Geographic holdout

Different geographic regions.

Sensor holdout

Different sensors where possible.

Query holdout

New query compositions.

Object holdout

Previously unseen object classes.

This tests whether SatQuery actually generalizes.

42. PHASE 33 — ISRO-SPECIFIC EVALUATION TRACK

This must have its own benchmark.

The central target should be the **Cartosat-2S optical + RISAT SAR paired setting described for the SIH problem**, rather than optimizing only for public datasets.

This is particularly important because the public model ecosystem is moving rapidly: Earth-OneVision now covers six sensor modalities and reports strong cross-modal results, while RemoteReasoner explicitly targets autonomous geospatial workflows.

Therefore:

Public benchmark superiority $\neq$ SIH superiority.

43. FUTURE-PROOFING FOR ISRO'S DATA ECOSYSTEM

Don't hard-code SatQuery around one sensor.

ISRO's current EO ecosystem spans Bhuvan and Bhoonidhi, and the latter provides access to a large archive of Indian and foreign remote-sensing data.

Moreover, ISRO has now released operational **NISAR S-band SAR products through Bhoonidhi from Cycle 25 onward**, beginning July 8, 2026.

So architecturally:

Sensor Adapter

- └─ Cartosat
- └─ RISAT
- └─ NISAR
- └─ Sentinel
- └─ Landsat
- └─ future ISRO sensors

But:

For SIH validation

Keep:

Cartosat-2S + RISAT

as the primary target.

44. PHASE 34 — RELIABILITY TESTING

Create deliberate failure cases.

Case A

Missing T2.

Expected:

Cannot perform temporal analysis.

Case B

Cloud-heavy optical.

Expected:

reduced optical confidence / use available complementary evidence.

Case C

Optical/SAR disagreement.

Expected:

conflicting evidence.

Case D

No queried object.

Expected:

no reliable detection.

Case E

Wrong CRS.

Expected:

validation failure.

Case F

Insufficient spatial resolution.

Expected:

limitation warning.

Case G

Out-of-distribution object.

Expected:

uncertain/unsupported.

45. PHASE 35 — COMPLEX QUERY SUITE

This should become SatQuery's signature benchmark.

Examples:

Query 1

“Find all buildings constructed after 2024.”

Query 2

“Find new buildings within 500 m of the river.”

Query 3

“Find new buildings within 500 m of the river where vegetation declined.”

Query 4

“Identify flood-affected buildings using optical and SAR evidence.”

Query 5

“Show what changed in this industrial area between the two dates.”

Query 6

“Which areas experienced vegetation loss and how far are they from roads?”

Now the system must compose multiple operations.

Your previous research explicitly identified these multi-step compositions as a potential signature ability.

46. PHASE 36 — OPTIMIZATION

Only after correctness.

Optimize:

- GPU reuse
- model caching
- async execution
- parallel branches
- tiling
- result caching
- vector simplification
- batch inference
- quantization

Architecture:

FastAPI

↓

Task queue

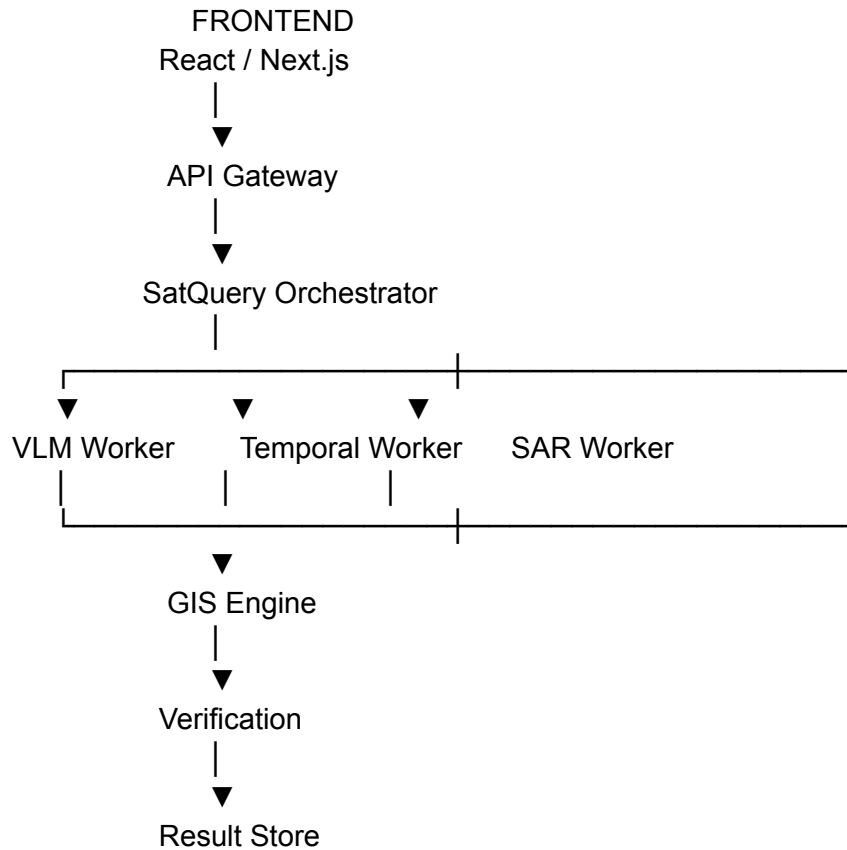
↓

Model workers

↓
GIS workers
↓
Verifier
↓
Result store

47. PHASE 37 — DEPLOYMENT ARCHITECTURE

Recommended:



Containerize:

frontend
orchestrator
model-service
gis-service

worker
database
object-store

48. PHASE 38 — SECURITY / DATA GOVERNANCE

For an ISRO-oriented system, add:

input sanitization
file validation
resource limits
model sandboxing
audit logs
data lineage
access control

Also avoid sending sensitive EO imagery to external APIs unless explicitly permitted.

49. PHASE 39 — FINAL USER EXPERIENCE

The interface should be deceptively simple.

User sees:

SATQUERY AI				
Ask a question about this data...				
[What changed in this region?]				
Upload	Optical	SAR	T1/T2	

After execution:

MAP	ANSWER
highlighted regions	183 changes detected

[Evidence] [Execution] [GeoJSON]

50. PHASE 40 — FINAL DIFFERENTIATOR LAYER

These are the **features** I now consider most worth pursuing.

★ Signature 1 — Query-to-Workflow Compiler

Natural language → executable EO workflow.

★ Signature 2 — Capability-Aware Model Router

The system knows what each specialist is good at.

★ Signature 3 — Sensor-Aware Reasoning

The system decides **which sensor evidence matters and why**.

★ Signature 4 — Compositional Geospatial Reasoning

One query can trigger many dependent analyses.

★ Signature 5 — Evidence Graph

Not just:

answer + confidence.

But:

Answer

↑

Evidence

↑

Models

↑

Sensors

↑

Images

★ Signature 6 — Self-Verification / Recovery

Failed validation can trigger:

- alternate model
 - alternate sensor
 - zoom
 - rerun
 - abstention.
-

★ Signature 7 — Actionable Geo-Output

text

+

map

+

mask

+

coordinates

+

statistics

+

GeoJSON

★ Signature 8 — Honest Uncertainty

The system can say:

“I cannot establish this from the available evidence.”

51. ONE MORE CAPABILITY I'D ADD

This is not essential for the first prototype, but potentially very valuable:

Evidence Recommendation

Instead of merely saying:

“Low confidence.”

SatQuery says:

“Confidence is low because optical imagery is cloud-obscured. A second temporal SAR observation would provide stronger evidence.”

So the system becomes:

Analyze



Detect uncertainty



Explain uncertainty



Recommend next evidence

That starts moving SatQuery from:

answer engine

toward:

investigation assistant.

52. MODEL SELECTION STRATEGY

I would not hard-code today's model list.

Instead:

Primary candidate pool

RS-VLM

- Earth-OneVision
- SkyEyeGPT
- VHM
- GeoChat
- EarthGPT-X
- RemoteReasoner
- RSThinker

EO backbone

- TerraMind
- SkySense V2
- SkySense++
- Prithvi-EO-2.0

Grounding

- GeoGround
- specialist RSVG systems
- Grounding DINO variants

Segmentation

- SAM2
- GeoGround
- RS-specialized segmentation systems

Temporal

- MIGRANT

- UniRS
- Prithvi
- SkySense family
- specialist change models.

This model ecosystem is already broader than the original blueprint, which is why we should make the architecture model-agnostic.

53. WHAT WE SHOULD ACTUALLY TRAIN

This is critical.

Don't train:

“A giant SatQuery VLM.”

Train/adapt:

1. Query planner

query

→ task specification

2. Router

task specification

→ model/tool selection

3. Evidence fusion

multiple outputs

→ combined evidence

4. Verification/calibration

result

→ trustworthy confidence

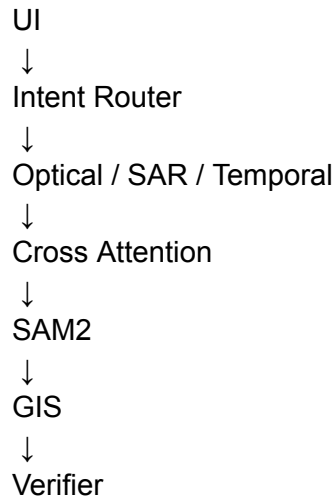
Potentially:

5. RS-VLM adapter

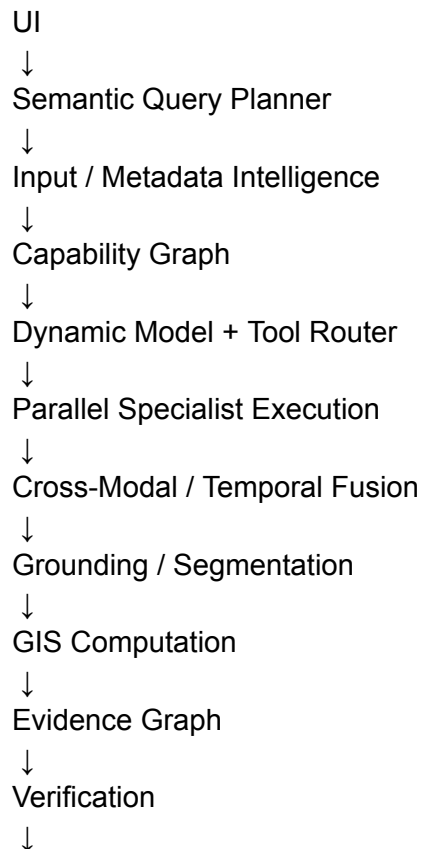
only after benchmarking reveals a meaningful weakness.

54. WHY THIS IS BETTER THAN YOUR CURRENT BLUEPRINT

Your current architecture is already strong conceptually:



But the evolved architecture should be:



Self-Correction



Calibrated Confidence



Actionable Geospatial Output



Execution Summary

That is a much more mature system.

55. PHASE-WISE TIMELINE

For a serious student team, I would structure roughly like this:

Phase	Duration	Priority
Requirements freeze	1–2 days	●
Model/Tool registry	2–4 days	●
Data ingestion	4–7 days	●
Input validation	3–5 days	●
Query parser	4–7 days	●
Workflow planner	5–7 days	●
Router	4–7 days	●
Single-image pipeline	4–7 days	●
Grounding + SAM2	4–7 days	●
GeoJSON/GIS	3–5 days	●
Temporal	5–10 days	●
Optical-SAR	5–10 days	●

Evidence engine	4–7 days	●
Verification	5–7 days	●
UI	5–10 days	●
Complex queries	5–7 days	●
Self-correction	3–5 days	●
Active perception	3–5 days	●
Benchmarking	7–10 days	●
Optimization	4–7 days	●
Final hardening	5–7 days	●

These overlap; this is **not** 100% sequential.

56. TEAM PARALLELIZATION

A good four/five-person team could work like this:

Person 1 — Agent/LLM

- query planner
- task specification
- router
- workflow engine.

Person 2 — EO/ML

- VQA
- grounding
- temporal
- model benchmarking.

Person 3 — Multimodal/GIS

- SAR
- optical-SAR
- Rasterio/GDAL
- GeoJSON
- indices.

Person 4 — Full-stack

- frontend
- map
- upload
- visualization
- execution trace.

Person 5 — Evaluation/infra

- datasets
 - benchmark
 - validation
 - model deployment
 - Docker
 - performance.
-

57. THE MVP BOUNDARY

Do **not** attempt every capability we've discussed in the first version.

MVP must contain:

- ✓ Natural-language query
- ✓ Input validation
- ✓ Agentic routing
- ✓ Single-image VQA
- ✓ Grounding
- ✓ SAM2 mask
- ✓ Bi-temporal change
- ✓ Optical + SAR

- ✓ Basic GIS
- ✓ Evidence
- ✓ Confidence
- ✓ GeoJSON
- ✓ Interactive map
- ✓ Execution summary

That directly covers the central requested workflow.

58. ADVANCED VERSION

After MVP:

- + active perception
 - + self-correction
 - + multi-model consensus
 - + richer temporal reasoning
 - + complex spatial queries
 - + retrieval
 - + evidence graph
 - + next-evidence recommendation
 - + broader sensors
-

59. RESEARCH VERSION

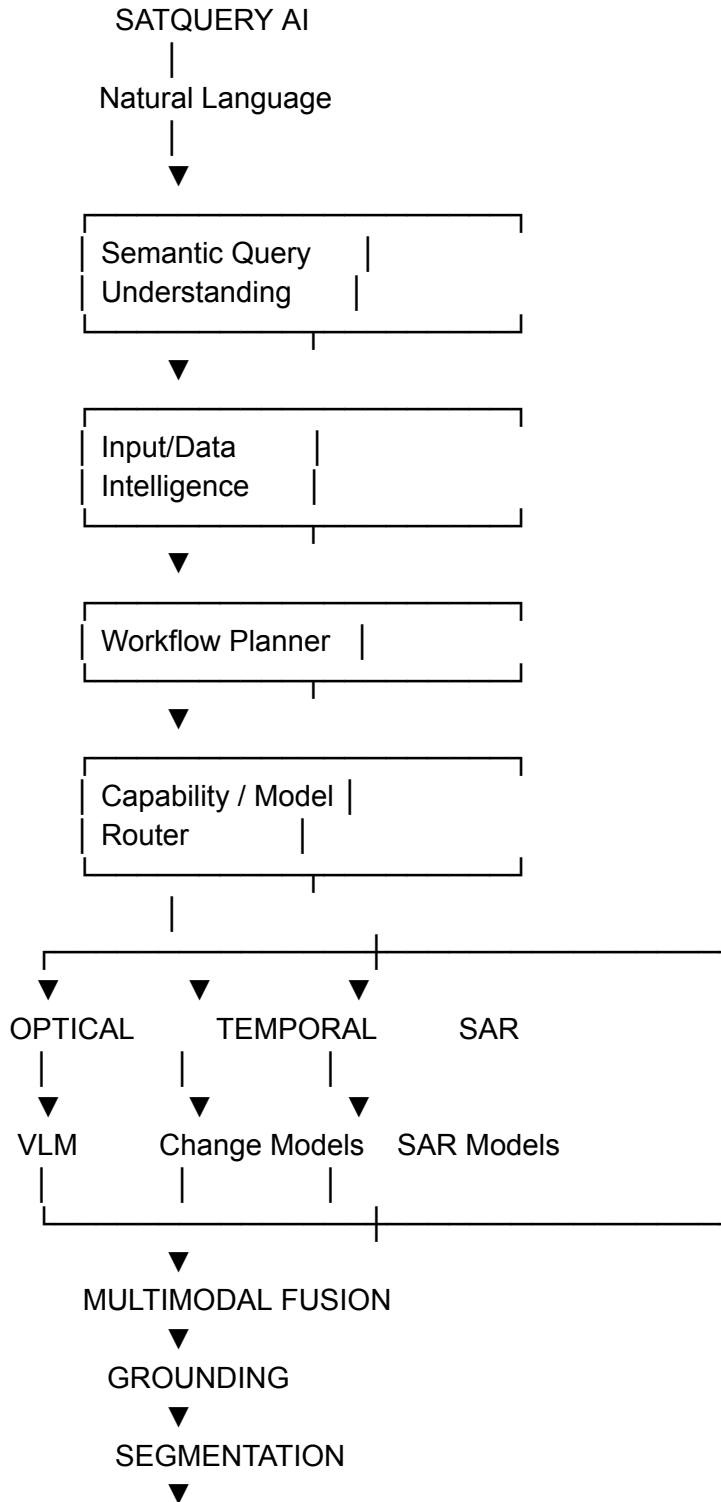
Longer term:

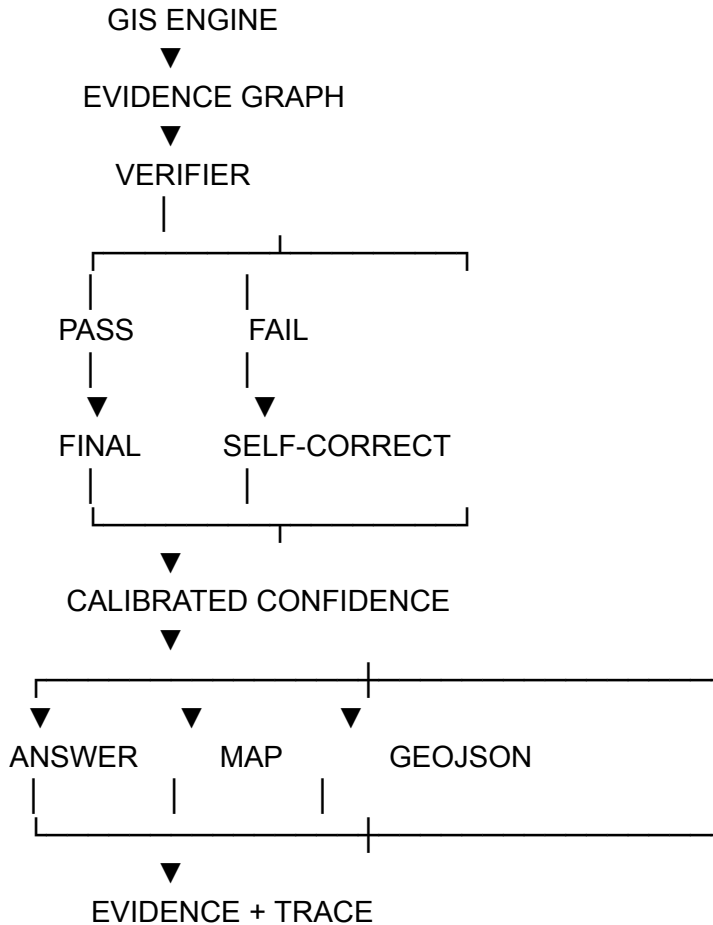
- + learned workflow planner
- + RL-based routing
- + learned multimodal fusion
- + adaptive sensor selection
- + retrieval over EO archives
- + multi-image/multi-date reasoning
- + autonomous investigation

RemoteReasoner already demonstrates that RL-based geospatial workflow reasoning is a viable research direction, so this is a natural future path rather than an arbitrary idea.

60. FINAL SATQUERY TARGET

The end-state becomes:





THE FINAL STRATEGIC FORMULA

After everything we've researched, I would define the project as:

SatQuery =

Strong existing EO models

+ intelligent query planning

+ dynamic specialist/tool routing

+ temporal reasoning

+ optical-SAR reasoning

+ **deterministic GIS analytics**

+ **grounding/segmentation**

+ **evidence fusion**

+ **verification**

+ **calibrated uncertainty**

+ **actionable geospatial outputs**

The important part is that **the middle layer is ours**.

We don't need to recreate every SOTA model. In fact, the current landscape strongly argues against that. Earth-OneVision already spans six sensor modalities and nine task categories; SkySense V2 is already a unified multimodal EO foundation model; RemoteReasoner already attacks unified geospatial workflow reasoning.

So our winning strategy is:

Use SOTA where SOTA already exists. Build the intelligence that coordinates it around the actual ISRO problem.

And your uploaded documents already contain several excellent seeds for that architecture: full-band/metadata-preserving ingestion, dynamic routing, optical/SAR/temporal streams, SAM2 cascading, geospatial projection, and verifier auditing. The next evolution is to make those components **semantic, measurable, model-agnostic, evidence-driven and experimentally validated**, rather than hard-coded.

The implementation order I would now lock

Requirements → data model → ingestion → validation → query planner → capability registry → router → single-image vertical slice → grounding/SAM2 → GIS → temporal → optical-SAR → evidence → verifier → self-correction → full agent → benchmark → optimization → final demo.

That is the roadmap I would use as the **master execution plan for SatQuery**, while keeping the ISRO-specific Cartosat-2S/RISAT workflow at the center rather than allowing the project to drift into a generic remote-sensing chatbot.